Empirical Software Engineering (2024) 29:47
https://doi.org/10.1007/s10664-023-10433-5

Evaluating the impact of flaky simulators on testing autonomous driving systems

Mohammad Hossein Amini[1] · **Shervin Naseri**[1] · **Shiva Nejati**[1]

Accepted: 30 November 2023 / Published online: 21 February 2024
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract
Simulators are widely used to test Autonomous Driving Systems (ADS), but their potential flakiness can lead to inconsistent test results. We investigate test flakiness in simulation-based testing of ADS by addressing two key questions: (1) How do flaky ADS simulations impact automated testing that relies on randomized algorithms? and (2) Can machine learning (ML) effectively identify flaky ADS tests while decreasing the required number of test reruns? Our empirical results, obtained from two widely-used open-source ADS simulators and five diverse ADS test setups, show that test flakiness in ADS is a common occurrence and can significantly impact the test results obtained by randomized algorithms. Further, our ML classifiers effectively identify flaky ADS tests using only a single test run, achieving F1-scores of 85%, 82% and 96% for three different ADS test setups. Our classifiers significantly outperform our non-ML baseline, which requires executing tests at least twice, by 31%, 21%, and 13% in F1-score performance, respectively. We conclude with a discussion on the scope, implications and limitations of our study. We provide our complete replication package in a Github repository (Github paper 2023).

Keywords Autonomous driving systems · Search-based testing · Machine learning and simulators

1 Introduction

Simulation-based testing is the technique of choice for at-scale verification of systems with high levels of autonomy, e.g., autonomous driving systems (ADS) (Raquel 2022; Ahlgren

Communicated by: Dietmar Pfahl

✉ Mohammad Hossein Amini
mh.amini@uottawa.ca

Shervin Naseri
snase041@uottawa.ca

Shiva Nejati
snejati@uottawa.ca

[1] University of Ottawa, Ottawa, ON, Canada

et al. 2021; Abdessalem et al. 2018; Borg et al. 2021). Simulators exercise very large numbers of system-usage scenarios that would be prohibitively expensive or time-consuming to enact in the real world. Many ADS rely on deep neural networks (DNNs) either partially or entirely. Testing DNN-enabled ADS using simulators can reveal failures that would remain undetected when testing DNNs individually and without embedding them into a closed-loop simulation environment (Haq et al. 2020, 2021). This highlights the significance of simulation-based ADS testing.

Recent studies have noted that ADS simulators can be flaky (Nguyen et al. 2021; Birchler et al. 2023), and an industry survey further emphasizes this flakiness as a major challenge to reproducibility in the field of robotic simulation (Afzal et al. 2021). Current simulation-based ADS testing methods either discard flaky tests to avoid inaccuracies (Birchler et al. 2023) or limit test scenario variables to reduce flakiness (Nguyen et al. 2021). These recent studies on ADS testing highlight the significance of addressing flaky tests in virtual environments and simulators since flaky tests lower the reliability of simulation-based testing for safety-critical systems (Birchler et al. 2023; Nguyen et al. 2021). However, to the best of our knowledge, there is no systematic study on the prevalence, impact, and potential mitigation strategies for test flakiness in simulation-based ADS testing.

In contrast, test flakiness has been widely studied in software code, where flaky tests are those exhibiting non-deterministic behavior by passing or failing over different runs when applied to the same codebase (Parry et al. 2021; Luo et al. 2014; Alshammari et al. 2021). These tests can be problematic and time-consuming, as they make it difficult to determine whether a code modification has caused a test to fail or if the failure is due to the test's flakiness.

In this paper, we present a systematic study on flakiness in simulation-based ADS testing. Though the investigated questions are relevant to all autonomous system simulators that exhibit non-determinism, we focus on ADS simulators due to their importance and widespread use. Our study aims to ultimately answer the following questions: *RQ1. How do flaky ADS simulations impact automated testing that relies on randomized algorithms?* and *RQ2. Can machine learning (ML) effectively identify flaky ADS tests while decreasing the required number of test reruns?* We answer these questions using two widely-used, open-source ADS simulators, CARLA (Dosovitskiy et al. 2017) and BeamNG (BeamNG.tech 2023), and based on five different ADS test setups. These test setups enable us to examine test flakiness across various ADS types while considering a diverse range of ADS input variables for testing purposes. These setups include (1) CARLA with its builtin PID-based ADS (Carla 2023); (2) CARLA with Pylot, a modular DNN-enabled ADS (Gog et al. 2021; Haq et al. 2022); (3) CARLA with Transfuser, an end-to-end DNN-enabled ADS (Github transfuser 2023; Haq et al. 2023), (4) BeamNG with its AI-engine (BeamNG.tech 2023), and (5) the BeamNG test setup from the tool competition track of the SBFT workshop (Github cyber 2023).

We present a generic framework for simulation-based testing of ADS, and use this framework to perform experiments that answer RQ1 and RQ2. The framework's test automation employs a basic random testing algorithm. We choose random testing as the basis of our experiments because most ADS testing research relies on metaheuristic and fuzz testing algorithms that are fundamentally rooted in random testing methods (Zeller et al. 2023; Luke 2013). We assess test outputs using quantitative fitness functions that are defined based on system requirements. These functions determine the extent to which a test satisfies or violates a given requirement. Fitness values are used to both guide search algorithms and generate Boolean verdicts, i.e., pass and fail results, for the requirements based on user-defined thresholds.

We define two distinct notions of flakiness for ADS testing: one based on Boolean verdicts and the other based on quantitative fitness values. The first notion, which we refer to as *hard*

flaky, aligns with the definition of flaky tests in the literature: A test is flaky if it passes and fails non-deterministically over multiple re-executions (Parry et al. 2021). The second notion, which we refer to as *soft flaky*, identifies a test as flaky when there are variations in the values of a fitness function used for testing.

Contributions We present the first study investigating the prevalence of flaky tests in ADS simulation-based testing and their impact on test results of ADS. Further, we study the effectiveness of machine learning classifiers in cost-effectively predicting flaky ADS tests and their ability to reduce the impact of flaky simulations on test results through a minimal number of test reruns. We address RQ1 and RQ2 (described earlier) using five distinct ADS test setups. Three of our test setups are adopted from the literature (Haq et al. 2022; Github transfuser 2023; Haq et al. 2023; Github cyber 2023). We developed the two others to augment our empirical results. Our findings for RQ1 and RQ2 are summarized below:

RQ1) Our results show that for our five test setups, 4%-68% of the generated tests exhibit notable variations in fitness values, indicating that all of our setups yield substantial soft flakiness. Further, the hard flaky rate for these setups ranges from 1% to 74%, with four out of five exhibiting a hard flaky rate exceeding 6% for at least one fitness function. To assess the impact of flakiness on randomized ADS testing, we compare a random testing algorithm that captures the best fitness value from multiple candidate test reruns with a baseline random testing algorithm that executes each candidate test once. Our results show that the former substantially outperforms the latter, as it computes significantly better fitness values and identifies considerably more failures, ranging from 12 to 888.

RQ2) To address the research question, we use three test setups, while excluding the Transfuser-based setup (Github transfuser 2023) due to its prohibitive computational cost, and the competition-based setup (Github cyber 2023) as a result of its simple test input design and virtually negligible hard flaky rate. We build ML classifiers that can effectively identify flaky ADS tests using only a single test run, achieving F1-scores of 85%, 82% and 96% for our three ADS test setups. Using a non-ML baseline that detects flaky tests based on two or more test reruns, we show that our classifiers significantly outperform this baseline, achieving F1-score improvements of 31%, 21%, and 13%, respectively.

In addition to answering the above two RQs, we present the following three lessons learned based on our findings: First, we confirm that the ADS test setup which is limited to checking the lane-keeping function (Github cyber 2023) shows the lowest rate of flaky tests. Second, based on our results, Pylot, which is a modular DNN-enabled ADS (Gog et al. 2021), produces considerably lower flaky tests compared to Transfuser, which is an end-to-end ADS (Github transfuser 2023). Third, the Carla simulator yields a lower flaky test rate compared to the BeamNG simulator.

It is important to clarify that our results should not be interpreted as criticisms of ADS simulators we consider in our study. CARLA and BeamNG are widely-used open-source simulators for ADS testing. They both have been used in several recent research on ADS testing, e.g., (Haq et al. 2022; Zhong et al. 2023; Haq et al. 2023). BeamNG has been used as a benchmark by the community (BeamNG.tech 2023). As we discuss in Section 5, many factors may contribute to the flakiness of simulators. Some of these factors, such as uncertainties in the physical models of simulators, are inherent to the physics-based design of both commercial and open-source simulators, and hence may be inevitable.

The main conclusion of our work is that the methods employed in the software engineering literature for assessing simulation-based ADS testing approaches may lack robustness and may be unreliable due to the flakiness of ADS simulators. Specifically, we rely on the number of individual failing test scenarios and the values of fitness functions as metrics to evaluate testing approaches. Our experiments, which are grounded in existing ADS test setups from

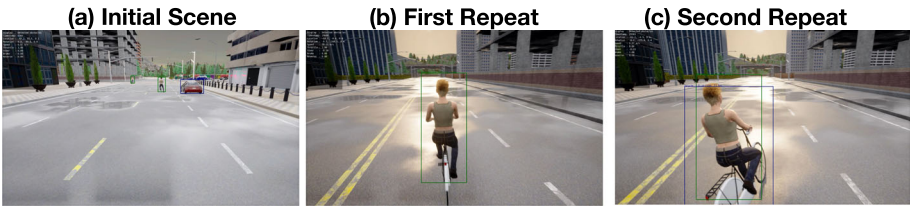


Fig. 1 Flakiness in ADS testing: scene (a) is an initial scene, while scenes (b) and (c) are taken from two separate re-executions of the same test input starting from scene (a). scenes (b) and (c) show the points of closest proximity between the ego car and the front bike. In scene (b), the accident is avoided, while scene (c) shows an accident

the literature (Haq et al. 2023, 2022; Github cyber 2023), demonstrate that the values of these metrics can exhibit significant variability between two versions of baseline random testing: one that executes each test only once and another that repeats each test multiple times. As we discuss in Section 5, we either need to consider restricted ADS test setups where the impact of flakiness is minimized, or we need to develop evaluation metrics that are not sensitive to flakiness.

Organization Section 2 motivates our work. Section 3 describes our generic ADS testing framework. Section 4 presents our empirical study. Section 6 compares with the related work. Section 5 outlines our observations and our discussions on the scope and implications of our study, and Section 7 concludes the paper.

2 Examples of Flaky Simulations

Figure 1 shows an example of flakiness in the CARLA simulator when executed with Pylot as the ADS. Figure 1(a) shows the initial scene, and Fig. 1(b) and (c) show the moments of nearest distance between the ego car and the bicycle ahead during two distinct re-executions, both originating from scene (a). These snapshots are captured by a camera mounted on the ego vehicle. In this example, Fig. 1(b) shows no accident between the ego car and the bike, whereas Fig. 1(c) shows an accident between the ego car and the bike.

The two re-executions shown in Fig. 1 individually look like valid and realistic simulations. Sometimes, re-executions of the same test input represent rare or even impossible situations in the physical world. For example, Fig. 2 shows another example of flakiness in ADS simulation-based testing obtained from CARLA executed with its PID-based ADS. While

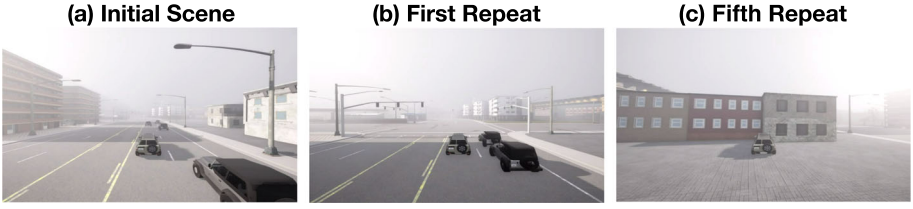


Fig. 2 Flakiness in ADS testing. Similar to Fig. 1, scene (a) is an initial scene, and scenes (b) and (c) are from two re-executions starting from scene (a). The re-execution related to (b) shows no significant event, while in the re-execution represented by (c), the ego-car shows unexplained behaviour and diverts from the road

the re-execution represented by the scene in Fig. 2(b) appears to be normal with no accidents or failures, the re-execution related to scene (c) shows the ego car displaying abnormal and unexplained instability, veering off the road, entering a parking lot, and crashing into a building. Through our study for RQ1 and RQ2 described in Section 1, we introduce an approach for evaluating and mitigating flakiness in ADS simulators. Our study uses an ADS simulation-based testing framework which is described in the next section.

3 ADS Simulation-Based Testing

Figure 3(a) shows an overview of ADS simulation-based testing that includes three elements: the test generator, the simulator, and the ADS which is the system under test (SUT). Below, we first provide background on each element. We then present a basic random testing algorithm for ADS. Finally, we define our two notions of flakiness in ADS testing.

Test generator The test generator produces test inputs to be executed by the simulator and receives, as test outputs, simulation logs and a simulation scenario. The terms *scene* and *scenario* are often used in the ADS testing literature and are defined as follows: *Scene* is a snapshot or frame in the simulation (Zhong et al. 2023), characterized by the properties of mobile objects, surrounding objects, roads and ambient conditions. *Scenario* is "the temporal development between several scenes in a sequence of scenes" (Ulbrich et al. 2015). Figure 4 shows a conceptual model detailing the test inputs and outputs that we designed for our ADS test setups. Specifically, a test input includes (1) a *configured* initial scene, (2) simulation duration, and (3) the time step duration, which is the time duration between each two consecutive scenes in a scenario. The configured initial scene includes the following information: (i) Mobile objects that always include a single ego vehicle and optionally some non-ego vehicles. For each mobile object, we typically specify the initial and end points, the target speed and the vehicle type. (ii) Surrounding objects such as buildings, traffic signs, parked cars, and pedestrians on the sidewalk. (iii) Layout information such as route maps and road shapes. (iv) Ambient conditions which include the weather condition and the time of the day. To be consistent with the ADS test setups adopted from the literature (Haq et al. 2022; Github transfuser 2023; Haq et al. 2023), pedestrians are static objects on the sidewalk. Due to the presence of non-ego vehicles, we can still test collision avoidance requirements using our test setups.

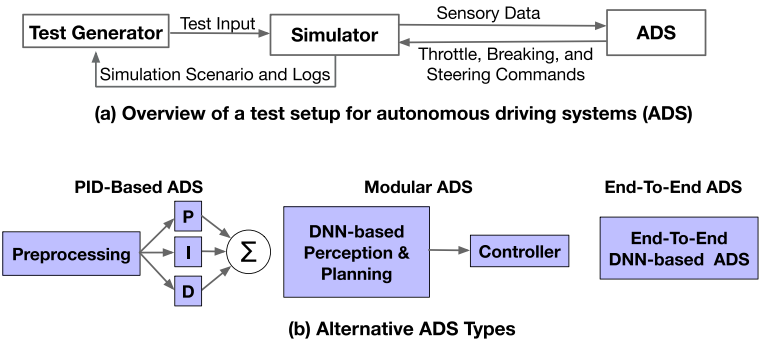


Fig. 3 ADS simulation-based testing

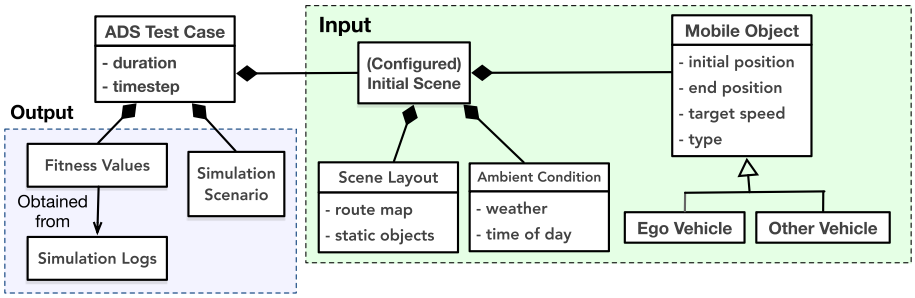


Fig. 4 A conceptual model detailing test inputs and outputs used in our ADS test setups

Simulator As Fig. 3 shows, a simulator takes an initial configured scene as input and generates a scenario based on the specified simulation time step and maximum duration.

ADS The ADS receives sensory data from the simulator and generates the throttle, breaking and steering commands. As Fig. 3(b) shows, we identify three different types of ADS based on their internal design: The first type primarily consists of a PID controller which is combined with a preprocessing component responsible for perception and planning. This component may use non-DNN-based machine learning (Carla 2023; Samak et al. 2020). The second type is a DNN-based modular design that integrates multiple DNNs into the perception and planning layer of ADS (Gog et al. 2021). The DNN outputs are then passed to a controller that generates throttle, braking, and steering commands. The third type is a DNN-based end-to-end design that uses a single DNN for vehicle control. The DNN directly generates commands to be sent to the simulator (Github transfuser 2023; Udacity 2016). In our experiments, we use instances of each of these three types of ADS that have been previously used in the ADS testing literature (Carla 2023; Gog et al. 2021; Github transfuser 2023; Haq et al. 2023; BeamNG.tech 2023; Github cyber 2023).

Fitness functions Fitness functions quantitatively estimate how close a test input is into violating the requirements of an ADS under test. For example, collision avoidance can be measured by calculating the minimum distance between the ego and non-ego cars or static objects. We define binary pass/fail verdicts to determine whether, or not, a test input violates a given requirement by comparing the fitness function value with a threshold. In our work, we adopt the thresholds from the prior studies that are often set at zero. For example, a test input violates the collision safety requirement if the minimum distance between the vehicles, or the ego car and an object is zero or near-zero.

Random testing for ADS Most ADS testing research relies a search-based testing (SBT) or fuzz testing (FT) algorithms (Zeller et al. 2023; Luke 2013; Matinnejad et al. 2017). SBT and FT aim to generate a limited and effective set of test cases using different meta-heuristics. Algorithm 1 shows a random testing algorithm which is the most basic form of SBT and FT. It randomly generates a test input i, and stores i's fitness as the optimal fitness in f_{opt} if it is better than the best fitness found so far. The algorithm continues until some stop condition is met. We assume optimal tests are those that have the lowest fitness values. Due to flaky simulations, Algorithm 1 re-executes each test input i for n times by calling the *Fitness(i,n)* routine on line 7. Algorithm 2 shows *Fitness(i,n)* that returns the most optimal fitness value (in our case, the lowest fitness value) among multiple re-executions of a given test input. As discussed in Section 1, we use Algorithm 1, a basic random testing, to assess the impact of flaky simulations on ADS testing.

Algorithm 1 Random search.

```
1  Input: n ← Number of re-execution of each test input
2  begin
3      i ← generateRandomTestInput()
4      f_opt ← Fitness(i, n)
5      while not (stop-condition) do
6          i ← generateRandomTestInput()
7          f' ← Fitness(i, n)
8          if f' < f_opt then
9              f_opt ← f'
10     return f_opt
```

Flakiness definitions for ADS testing Using quantitative fitness values, flakiness can be measured in two ways: (1) soft flakiness, reflecting variations in fitness values across test re-executions, and (2) hard flakiness, indicating changes in pass/fail verdicts based on different test re-executions. Hard flakiness resembles flakiness in software code testing, while soft flakiness is an additional concept stemming from quantitative fitness functions. By definition, hard flakiness implies soft flakiness, but not vice versa.

Definition 1 Let i be a test input, and let $f_1 \ldots, f_n$ be the fitness values obtained from multiple executions of i. We define soft flakiness of i, denoted by SF_i, and hard flakiness of i, denoted by HF_i as follows:

$SF_i = \max(\{f_1, \ldots, f_n\}) - \min(\{f_1, \ldots, f_n\})$

$HF_i = \exists f, f' \in \{f_1, \ldots, f_n\} : f > thr \wedge f' \leq thr$

where the threshold *thr* determines the pass/fail verdict.

Note that SF_i is a quantitative measure, while HF_i is Boolean. We use both notions of flakiness in our empirical evaluation presented in the next section.

4 Empirical Evaluation

In this section, we study test flakiness in simulation-based ADS testing by answering the two research questions we motivated in Section 1, which are re-stated below:

Algorithm 2 Fitness(i, n).

```
1  Input: i ← Test input
2  Input: n ← Number of re-executions
3  begin
4      s ← simulate(i)
5      f_1, f_opt ← calculateFitness(s)
6      for j ∈ {2, 3, ..., n} do
7          s ← simulate(i)
8          f_j ← calculateFitness(s)
9          if f_j < f_opt then
10             f_opt ← f_j
11     return f_opt
```

RQ1 *How do flaky ADS simulations impact automated testing that relies on randomized algorithms?* We study the impact of flaky ADS simulations on automated testing using three sub-research questions. First, we use the following sub-research question to determine the frequency of flaky tests:

RQ1-1 *What is the frequency of flaky tests in ADS simulation-based testing?* We apply to five ADS test setups (i.e., Carla PID, Carla Pylot, Carla Transfuser, BeamNG AI, and BeamNG Competition) the random testing algorithm, Algorithm 1. To report the soft and hard flakiness metrics (Definition 1) for the randomly generated test, each test is re-executed multiple times (i.e., in Algorithm 1, we set $n > 1$).

A possible strategy is to discard test re-executions exhibiting extreme abnormalities. For example, Fig. 2(c) shows a test re-execution with extreme abnormality, while the test re-execution in Fig. 2(b) looks normal. Eliminating abnormal cases may mitigate flakiness if the majority of flaky ADS tests consist of occasional major abnormalities among predominantly normal and consistent re-executions. However, if most re-executions are normal but still exhibit noticeable variations, such as the example in Fig. 1, this approach will not be effective. We present the following sub-research question to determine if most flaky tests are, in general, categorized by consistent and normal re-executions with occasional extreme abnormalities, or if they exhibit variations but with few extreme abnormalities:

RQ1-2 *Do individual re-executions of flaky ADS test inputs represent normal scenarios?* We examine re-executions of flaky tests to determine whether they represent extreme abnormalities, or whether, despite generating different fitness values, they still represent normal driving scenarios.

In addition to evaluating the prevalence of flaky tests (RQ1-1) and examining the normality of the observed variations over test reruns (RQ1-2), we evaluate the scale of the impact of flaky tests on ADS testing through the following question:

RQ1-3 *Can the variations caused by flaky tests substantially impact the results of ADS testing algorithms?* We compare the performance of random testing, a baseline ADS testing algorithm, for two different configurations: (1) each test input candidate is executed once (i.e., Algorithm 1 with $n = 1$), and (2) each test input candidate is executed multiple times (i.e., Algorithm 1 with $n > 1$). For a deterministic test setup, rerunning the same candidate tests multiple times should have no impact. While the presence of flaky tests means that rerunning the same test is likely to impact the results of ADS testing algorithms, ideally, the magnitude and significance of this impact should be minimal. We evaluate the significance of observed variations using effect sizes and statistical tests for the two widely-used metrics in the SBT and FT literature (Abdessalem et al. 2018; Haq et al. 2022; Zeller et al. 2023): the number of detected failures and the optimality of the fitness values calculated by each algorithm.

RQ2 *Can machine learning (ML) effectively identify flaky ADS tests while decreasing the required number of test reruns?* Since both soft and hard flakiness may impact ADS testing, and since soft flaky is a weaker notion, we focus on using ML to predict soft flakiness for ADS test inputs. To do so, we use the set of input variables and fitness values as features for learning. These features are typically available for any ADS test setup regardless of the specific simulator or the ADS controller used. We experiment with different subsets of input variables to identify the most relevant and informative features for predicting flakiness. In addition, we consider two alternative feature designs to capture fitness functions. The first design uses individual fitness values obtained from single test runs, while the second design

uses differences of fitness values obtained from multiple re-executions of the same tests. The first design requires less effort as it only needs one execution of each test input, while the second design needs multiple executions. We refer to an ML model built based on the first design as single-test-execution classifier (STEC) and that built based on the second design as multi-test-execution classifier (MTEC). We assess both types of classifiers by studying the trade-off between their prediction accuracy and the number of (re-)executions needed to produce their input features. We also compare their performance with a baseline that does not rely on machine learning.

Predicting flakiness in ADS testing may fulfill two different goals. The first goal concerns with detecting flaky test inputs, while the second aims to improve ADS testing by identifying failure scenarios with the most optimal (lowest) fitness values. We examine these goals through the following sub-questions:

RQ2-1 *How accurate and cost-effective are machine learning classifiers in predicting flaky ADS tests?* We evaluate the precision, recall and F-1 score of alternative classification techniques in predicting flaky ADS test inputs. We consider the two feature designs (i.e, STECand MTEC) to build classifiers. Further, we compare the classifiers with a non-ML baseline.

RQ2-2 *Can machine learning classifiers improve the performance of ADS testing by requiring a limited number of test reruns?* For this question, we modify Algorithm 2 to develop Algorithm 3, which uses predictions of ML classifiers (i.e., STECand MTEC) to minimize the number of required reruns for a candidate test. In other words, Algorithm 3, instead of re-running each test an equal number of times, reruns a test only until we can infer it is flaky. Following the first fitness calculation, STECpredicts, in line 6 of Algorithm 3, whether test i is flaky or not, based on the fitness value obtained from the single simulation in line 4. The algorithm proceeds with the while-loop in lines 8-15 only if STEClabels test i as flaky. In each iteration, the while-loop re-executes test i, computes a new fitness value, and calculates the maximum difference (i.e., δ) among fitness values obtained for test i so far. It then uses MTECto predict flakiness based on δ. Note that inside the loop, given the availability of multiple test executions and the maximum fitness difference, we use MTECinstead of STEC. The while-loop runs up to n iterations or until MTEClabels test i as non-flaky. The algorithm concludes by returning the optimal fitness value (f_{opt}).
While Algorithm 2 runs each candidate test n times, Algorithm 3 likely runs several candidate tests only once or fewer than n times. Our goal is to experiment with different ML classifiers to determine if Algorithm 3 can obtain optimal fitness values that are close to those obtained by Algorithm 2 while requiring far fewer test reruns.

4.1 ADS Test Setups

Our study uses the CARLA (Dosovitskiy et al. 2017) and BeamNG (BeamNG.tech 2023) simulators together with four different ADS: (1) CARLA's PID-based traffic manager (Carla 2023); (2) Pylot, a DNN-enabled modular ADS (Gog et al. 2021); (3) Transfuser, a DNN-enabled end-to-end ADS (Github transfuser 2023); and (4) BeamNG's AI-driven default ADS (BeamNG.tech 2023). We develop five different instances of the ADS test setup of Fig. 3 by combining these simulators and ADS. Three of these setups use CARLA as the simulator, with traffic manager, Pylot, and Transfuser as the respective ADS. In the remainder of this paper, we refer to these three as PID, PYLOT and TRAN, respectively. For the test inputs and fitness functions of PYLOT and TRAN, we rely on the test generators provided

Algorithm 3 fastFitness(i, n).

```
1  Input: i ← Test input
2  Input: n ← Maximum Number of re-executions
3  begin
4      s ← simulate(i)
5      f_1, f_opt ← calculateFitness(s)
6      flaky =STEC(i, f_1) // single-test-execution classifier
7      j ← 1
8      while (flaky ∧(j < n)) do
9          j ← j + 1
10         s ← simulate(i)
11         f_j ← calculateFitness(s)
12         if f_j < f_opt then
13             f_opt ← f_j
14         δ = max({f_1, ..., f_j}) − min({f_1, ..., f_j})
15         flaky =MTEC(i, δ) // multi-test-execution classifier
16     return f_opt;
```

in the literature (Haq et al. 2022; Github transfuser 2023; Haq et al. 2023). We ensure the consistency of the test generators for these two setups. The details as to how we adopted the test inputs and fitness functions for these two setups are available in our repository (Github paper 2023). For PID, we developed, from scratch, a test generator that produces test inputs and computes fitness functions compatible with those used for PYLOT and TRAN. For the other two test setups, we use BeamNG as the simulator together with its AI-based ADS, but we use two different test generators: one that aligns with the test generators for PID, PYLOT and TRAN, and another based on the setup for the Cyber-Physical Systems Testing Tool Competition track of the SBFT workshop (Github cyber 2023). We refer to these two setups as BEAMNG and COMP, respectively. The characteristics of our ADS test setups are summarized in Table 1. Below, we briefly describe the test inputs and fitness functions used in our five test setups.

Test inputs The test inputs for PID, PYLOT, TRAN and BEAMNG conform to the conceptual model in Fig. 4. For COMP, we follow the competition website's test input design (Github cyber 2023), which only includes information about the route map. COMP test inputs do not include any non-ego vehicles, static objects or any information about the weather or the time of day. The PYLOT, TRAN and COMP repositories (Gog et al. 2021; Github transfuser 2023; Haq et al. 2022; Github cyber 2023) already include a random testing baseline implemented. This random testing samples test inputs within their specified ranges assuming that each input variable has a uniform distribution. We used these already implemented random baselines for our experiments, and implemented similar random testing algorithms for PID and BEAMNG, i.e., the test setups implemented in this paper. The implementation of the random testing algorithms is available in our replication package [2].

Fitness functions For PID, PYLOT, TRAN and BEAMNG, we evaluate the test outputs against four ADS requirements:

R1: *The ego car should remain within its lane, only deviating when intentionally changing lanes.*
R2: *The ego car should always maintain a safety distance from other vehicles.*
R3: *The ego car should always maintain a safety distance from the sidewalk and static objects.*

Table 1 Characteristics of the ADS test setups used in our experiments

SETUP ID	SIMULATOR	ADS	TEST INPUTS	FITNESS FUNCTIONS	SOURCE
PID	CARLA	CARLA PID	Fig. 4	$F_1 \ldots F_4$	Our replication package (Github paper 2023)
PYLOT	CARLA	Pylot	Fig. 4	$F_1 \ldots F_4$	Gog et al. (2021)
TRAN	CARLA	Transfuser	Fig. 4	$F_1 \ldots F_4$	Github transfuser (2023); Haq et al. (2022)
BEAMNG	BeamNG	BeamNG AI	Fig. 4	$F_1 \ldots F_4$	Our replication package Github paper (2023)
COMP	BeamNG	BeamNG AI	A single-lane road	F_1	Github cyber (2023)

R4: *The ego car should reach the specified destination within the maximum simulation time duration.*

We define four fitness functions, referred to as F_1, F_2, F_3 and F_4, respectively, to evaluate these four requirements:

F_1 measures the number of lane invasions not followed by a lane change.
F_2 measures the minimum distance between the ego and non-ego cars;
F_3 measures the minimum distance between the ego car and static objects or sidewalk; and
F_4 measures the distance to the destination position.

For PYLOT, TRAN and COMP, the implementations of the above four fitness functions are respectively taken from the repositories provided by the sources of these ADS test setups (Haq et al. 2022, 2023; Github cyber 2023). The Comp setup (Github cyber 2023) uses a one-lane road. For this setup, a lane invasion is computed based on the out-of-bound (OOB) distance that is measured as the average of the differences between the lane's width and the distance from the ego car to the lane's center across all time steps of the simulation scenario. For PYLOT (Haq et al. 2022) and TRAN (Haq et al. 2023), F_1 is calculated similarly to OOB. However, a distinction is made considering that the map for PYLOT and TRAN is a two-lane road intersection, and the ego car must make a lane change to reach its destination. In this context, one lane invasion followed by a lane change is deemed intentional and is not included in the computation of F_1. For BEAMNG and PID that are implemented by the authors, we have adopted this latter implementation for F_1 since the map of BEAMNG and PID is similar to that of PYLOT and TRAN. We have also adopted the implementations of F_2, F_3 and F_4 from the PYLOT (Haq et al. 2022) setup and verified their consistency with those in the Tran setup (Haq et al. 2023). Finally, for each fitness function, we used the thresholds provided by the source repositories for PYLOT, TRAN and COMP (Haq et al. 2022; Github cyber 2023; Github transfuser 2023). The implementation of the four fitness functions and the threshold used for each fitness function are detailed in our replication package (Github paper 2023).

4.2 RQ1-1 Results

This research question aims to identify the frequency of flaky tests in ADS simulation-based testing. For this research question, we apply random testing (Algorithm 1) to each of our five test setups. We set the number of test reruns, i.e., parameter n of Algorithm 1, to ten based on our preliminary experiments that were aimed at revealing flakiness in our different test setups. We generate 1000 random tests for each of PID, PYLOT, BEAMNG, and COMP, and 100 random tests for TRAN. We ensure the generation of diverse and unique test inputs as per the definitions of test input diversity in ADS testing (Abdessalem et al. 2018; Zhong et al. 2023). Each execution of PID, PYLOT, TRAN, BEAMNG and COMP on average takes 1.5min, 5.6min, 12min, 2min, and 1min, respectively. In total, we performed 10, 000 simulations for each of PID, PYLOT, BEAMNG, and COMP, and 1, 000 simulations for TRAN. All experiments were conducted on a machine with a 2.5 GHz Intel Core i9 CPU and 64 GB of DDR4 memory.

Metrics We calculate the soft flaky *SF* and hard flaky *HF* measures (see Definition 1) for every fitness F_1, F_2, F_3 and F_4 of each setup individually. Recall that the COMP setup has only one fitness function (F_1).

Results Table 2 presents the soft flakiness results for each fitness function of each setup. Recall from Definition 1 that soft flakiness captures the variations in the fitness function values and is denoted by SF_j for a test input j. Further, we denote by $MaxSF_{s,F}$ the largest soft flaky value among all the test inputs generated for setup s and fitness function F, and

by $R_{s,F}$ the value range of the fitness function F for setup s. The third and fourth columns from the left of Table 2, respectively, show the fitness range (R) and the max soft flaky value ($MaxSF$) for each fitness function of each setup. Among the 17 rows of Table 2, in twelve rows, the maximum soft flaky is equal to the fitness range; in three rows, $MaxSF$ is at least 93% of the fitness range; for one case, $MaxSF$ is at least 80% of the fitness range; and only in one case (i.e, F_3 of PYLOT), $MaxSF$ is only 20% of the maximum fitness range. These results show that maximum soft flakiness can be as high the fitness function range in several cases.

For each setup s and each fitness function F, we compute the ratio of soft flakiness for test input j as $SF_j/MaxSF_{s,F}$. The fifth to ninth columns from the left of Table 2 show, for each fitness function and each setup, the number of test inputs with soft flaky ratios that fall into different intervals, ranging from $[0-1\%]$ up to $(40\%-100\%]$. For example, out of 1000 test inputs generated for the fitness F_1 of PYLOT, 443 have soft flaky ratios within the 0-1% range. But, for the same fitness function and the same setup, the soft flaky ratio for 230 test inputs is more than 5%. Considering the soft flaky ratio of more than 5% as non-negligible, at least 50% of the tests in the six rows highlighted blue show non-negligible soft flakiness, and at least 10% of the tests in the five rows highlighted green show non-negligible soft flakiness.

Table 3 shows the number of hard flaky tests among all the tests generated for each fitness function of each setup. Recall from Definition 1 that hard flakiness is Boolean. As the table shows, the percentages of hard flaky tests for PID are between 4% to 16% for its four fitness functions. For PYLOT, 1% and 6% of the tests are, respectively, hard flaky for F_1 and F_2. For F_1 and F_2 of TRAN, there are, respectively, 40% and 4% hard flaky tests. BEAMNG yields the most hard flaky tests ranging between 22% to 74%, while for COMP and its single fitness function F_1, we have a low hard flaky rate (1%).

We do not conduct further experiments with the TRAN setup due to its prohibitive computational cost. The COMP setup is also excluded from the subsequent research questions due to its simple inputs and having only one fitness function. In addition, as Tables 2 and 3 show, COMP exhibits relatively low flakiness. We discuss the relation between the characteristics of our test setups and flakiness in Section 5.

The answer to RQ1-1 is that between 4% and 98% of the generated tests across our five test setups exhibit noticeable variations in their fitness values, indicating a significant presence of soft flakiness. At least one of the fitness functions of PID, PYLOT, TRAN, BEAMNG, and COMP exhibit hard flaky rates of 16%, 6%, 40%, 74%, and 1%, respectively.

4.3 RQ1-2 Results

To identify variations in flaky tests, this RQ requires us to inspect simulations from test reruns. To reduce discrepancies caused by potential differences between the setups and the simulators, we focus on PID and PYLOT, as both employ CARLA. We randomly select, from the tests generated for RQ1-1, 20 tests for PID and 20 tests for PYLOT. The selection was made such that for each fitness function of each setup, we selected at least five tests that exhibit non-negligible soft flakiness (i.e., a soft flakiness ratio higher than 5%). Two co-authors then watched the videos of ten reruns of the selected tests to identify variations in the ego vehicle's behavior that contributed to the flakiness of the fitness values. In all the selected tests, we identified visually-visible variations in the behaviour of the ego vehicle, leading to the differences in the fitness values. After the initial viewing, the co-authors re-examined the videos, compared the observed variations, and summarized them in Table 4. Note that the

Table 2 Soft flakiness (SF) results for each fitness function of our five test setups

Setup	F.	*R*	*MaxSF*	[0%–1%]	(1%–5%]	(5%–10%]	(10%–40%]	(40%–100%]
PID	F_1	2	1.99	79% (783)	14% (140)	2.4% (24)	5% (49)	0.4% (4)
	F_2	2	1.79	6% (58)	30% (297)	16% (157)	26% (260)	23% (228)
	F_3	2	2	77% (765)	4% (37)	2% (22)	15% (151)	3% (25)
	F_4	2	2	87% (869)	9% (94)	2% (16)	2% (15)	1% (6)
PYLOT	F_1	1.59	1.26	44% (433)	34% (337)	12% (122)	8% (82)	3% (26)
	F_2	2.98	2.75	90% (898)	4% (36)	1% (10)	4% (37)	2% (19)
	F_3	0.90	0.18	96% (963)	0% (0)	1% (7)	1% (10)	2% (20)
	F_4	0.96	0.96	90% (899)	1% (12)	1% (6)	4% (41)	4% (42)
TRAN	F_1	1.11	1.11	0% (0)	6% (6)	21% (21)	43% (43)	30% (30)
	F_2	142	142	87% (87)	3% (3)	1% (1)	7% (7)	2% (2)
	F_3	0.3	0.3	46% (46)	50% (50)	0% (0)	0% (0)	4% (4)
	F_4	0.08	0.08	0% (0)	2% (2)	20% (20)	66% (66)	12% (12)
BEAMNG	*F1*	2	2	33% (333)	0% (0)	0% (0)	0% (0)	67% (667)
	F2	2	2	34% (337)	20% (201)	18% (177)	28% (276)	1% (9)
	F3	2	2	24% (240)	8% (77)	2% (24)	34% (335)	32% (324)
	F4	2	2	26% (263)	6% (62)	7% (68)	24% (241)	36% (366)
COMP	*F1*	2.8	2.8	57% (570)	30% (302)	5% (51)	7% (71)	0.6% (6)

The third and fourth columns from the left, respectively, show the fitness function range (R) and the maximum SF value ($MaxSF$). The fifth to ninth columns from the left show intervals for soft flaky (SF) ratios with 0% meaning no soft flaky and 100% being the maximum SF value (i.e., $MaxSF$). For example, the sixth column from the left indicates the number of test inputs with a soft flaky ratio ($SF/MaxSF$) within the ($\mathbf{1\% - 5\%}$] interval. Considering the soft flaky ratio of more than 5% as non-negligible, rows with a minimum of 50% and 10% of tests exhibiting non-negligible soft flakiness are highlighted in blue and green, respectively

Table 3 Number of hard flaky tests for our ADS test setups

Fitness	PID	PYLOT	TRAN	BEAMNG	COMP
F_1	$60 \approx 6\%$	$12 \approx 1.2\%$	$40 \approx 40\%$	$669 \approx 66\%$	$9 \approx 1\%$
F_2	$66 \approx 6\%$	$64 \approx 6\%$	$4 \approx 4\%$	$744 \approx 74\%$	–
F_3	$163 \approx 16\%$	0	0	$328 \approx 32\%$	–
F_4	$40 \approx 4\%$	0	0	$227 \approx 22\%$	–

variations describe the differences observed among multiple runs of the same test input. For example, "*The ego vehicle hits other vehicles non-deterministically.*" indicates that, in some runs of a given test input the ego vehicle collides with other vehicles, while in some other runs, it does not. The videos from which the variations in Table 4 are extracted are available online (Github paper 2023).

We classify variations into three types: (I) Infeasible scenarios that violate fundamental physics principles. (II) Significant deviations in the ADS controller's expected behavior due to incorrect set-points, input frequencies exceeding plant bandwidth, or noise-corrupted inputs from faulty sensors (e.g., the example in Fig. 2). (III) Scenarios that slightly differ from one another but are normal and respect both physical laws and controller behavior (e.g., the example in Fig. 1). Table 4 presents the types of variations as well as the frequencies of the occurrence of these variation types in the analyzed videos. Among the three variation types, only scenarios with type I variations should be excluded from ADS test results since they represent flawed scenarios. Type II and III, however, represent meaningful scenarios

Table 4 Inconsistencies across repeated runs of flaky tests

Setup	Variation	Type	Freq. (out of 40)
PID	The ego vehicle suddenly drives to the pavement.	II	1
	The ego vehicle suddenly starts to turn around itself.	II	8
	Minor variations in traffic light timing impact the behavior of the ego vehicle.	III	4
	The streets disappear and the ego vehicle is no longer on the ground.	I	1
	Non-ego vehicles become unstable and impact the ego vehicle.	II	2
	The ego vehicle hits other vehicles non-deterministically.	III	2
PYLOT	The ego vehicle stops behind a red traffic light (or behind a vehicle that is waiting for a red traffic light) and fails to start moving after the light turns green.	III	5
	The ego vehicle stops for no apparent reason while turning.	III	2
	The ego vehicle stops for no apparent reason while driving on a straight road.	III	1
	The ego vehicle hits other vehicles non-deterministically.	III	7
	Randomness in the ego vehicle's steering for no reason.	III	3
	Randomness in the ego vehicle's behaviour when close to a non-ego vehicle.	III	4

and may help with revealing actual failures and with fault detection. Hence, they should not be disregarded.

The answer to RQ1-2 is that, among 40 randomly selected flaky test samples, only one sample yield flawed test reruns of type I. The reruns of the rest of the samples represent variations that are due to unstable controllers (type II) or minor differences in the behavior of the ego vehicle (type III). These variations should not be discarded and should be further investigated to determine whether, or not, they indicate valid failures in the ADS behaviour.

4.4 RQ1-3 Results

This research question aims to evaluate the *scale* of the impact of flaky tests the performance of ADS testing. We apply the random testing algorithm (Algorithm 1) to PID, PYLOT and BEAMNG. We run Algorithm 1 once with $n = 1$ and once with $n = 10$. We refer to the former as $RS_{n=1}$ and to the latter as $RS_{n=10}$. We run each of them for 50 iterations and record the best fitness function value (f_{opt}) at each iteration. Note that $RS_{n=1}$ does not account for test flakiness while $RS_{n=10}$ records, for each test input, the best (or lowest) fitness value obtained based on ten re-executions. To statistically compare the results, we repeat each 50-iteration run of these algorithms 20 times.

Metrics We compare $RS_{n=1}$ and $RS_{n=10}$ using two metrics commonly used in the literature to assess ADS testing algorithms (Harman and McMinn 2010; Abdessalem et al. 2018): (1) The number of failure revealing tests generated by each algorithm, and (2) the fitness values of the tests generated by each algorithm, where the algorithm that produces more optimal fitness values is considered better.

Results Table 5 compares the number of failure-revealing tests identified by each of $RS_{n=10}$ and $RS_{n=1}$. A test is failure-revealing with respect to a fitness function if the test's fitness value falls below the nominal threshold for that function (see Section 3). As Table 5 shows, for all the fitness functions of PID and BEAMNG, and for two fitness functions of PYLOT, $RS_{n=10}$ detects more failures than $RS_{n=1}$. For functions F_3 and F_4 of PYLOT neither $RS_{n=10}$ nor $RS_{n=1}$ identifies any failures.

Figure 5(a-c) show the trends for the averages and 95% confidence intervals of best fitness values obtained from 20 runs of $RS_{n=1}$ and $RS_{n=10}$ over 50 iterations for four fitness functions of PID, PYLOT and BEAMNG, respectively. The distributions of the final best fitness values, i.e., the fitness values at iteration 50, obtained from $RS_{n=1}$ and $RS_{n=10}$ are available online (Online 2023). Table 6 compares these distributions using the Wilcoxon signed-rank test (Capon 1991) and the Vargha Delaney $\hat{A}_{12}$ effect size (Vargha and Delaney 2000). As shown in Table 6, $RS_{n=10}$ outperforms $RS_{n=1}$ significantly in finding more optimal fitness

Table 5 Comparing the numbers of failure-revealing tests obtained by $RS_{n=10}$ and $RS_{n=1}$ for different ADS test setups

Fitness	PID	PYLOT	BEAMNG
	$RS_{n=10} - RS_{n=1}$	$RS_{n=10} - RS_{n=1}$	$RS_{n=10} - RS_{n=1}$
F_1	611 − 203	12 − 0	669 − 240
F_2	645 − 0	66 − 19	744 − 712
F_3	888 − 122	0 − 0	328 − 325
F_4	489 − 225	0 − 0	227 − 129

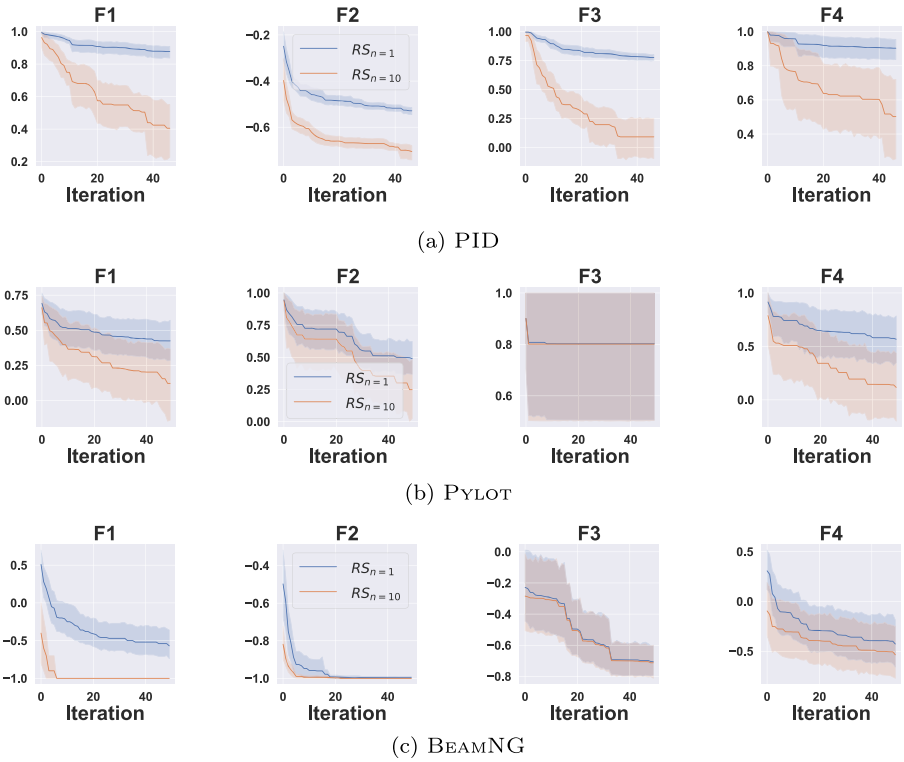


Fig. 5 The averages and 95% intervals of the best fitness values obtained by 20 runs of $RS_{n=1}$ and $RS_{n=10}$ over 50 iterations for four fitness functions of PID, PYLOT and BEAMNG

values in all the cases. Further, the comparison yields a large effect size for the four fitness functions of PID, three fitness functions of BEAMNG, and two fitness functions of PYLOT.

The answer to RQ1-3 is that, for our three ADS test setups, a random testing algorithm that reruns ADS tests multiple times significantly outperforms an algorithm that runs ADS tests once with large effect sizes for at least two fitness functions of each test setup. In addition, the former algorithm detects significantly more failures, ranging from 12 to 888, and yields fitness values that are significantly more optimal.

Table 6 Statistical test and effect size, Wilcoxon p-value and Vargha-Delaey $\hat{A}_{12}$ results comparing the distributions of best fitness values obtained from $RS_{n=10}$ and $RS_{n=1}$ at the last iteration of Fig. 5

Fitness	PID		PYLOT		BEAMNG	
	p-value	$\hat{A}_{12}$	p-value	$\hat{A}_{12}$	p-value	$\hat{A}_{12}$
F1	$7.50e-10$	0.98 (L)	$6.62e-156$	0.76 (L)	7.53e − 10	0.98 (L)
F2	$7.50e-10$	0.96 (L)	$8.78e-141$	0.76 (L)	7.29e − 10	0.76(L)
F3	$7.46e-10$	0.94 (L)	$1.03e-104$	0.62 (S)	7.54e − 10	0.58 (S)
F4	$7.50e-10$	0.89 (L)	$6.85e-144$	0.70 (M)	7.53e − 10	0.79 (L)

4.5 RQ2-1 Results

This research question evaluates if machine learning classifiers can be used to predict flaky ADS tests in an accurate and cost-effective manner. For this research question, we develop ML classifiers to predict flaky test inputs. As discussed earlier in Section 4, we focus on using ML to predict soft flakiness. To develop training data for predicting soft flakiness, we label a test input as flaky if it yields a soft flaky ratio higher than 5% for at least one fitness function (see Table 2 for the soft flaky ratio results). Through additional experiments using thresholds of 10% and 20%, we have verified that our findings for RQ2-1 are not sensitive to the 5% threshold (see our supplementary material available online Online 2023).

As discussed earlier, we explore two alternative input feature designs for our classifiers: STEC, which utilizes test inputs and individual fitness values, and MTEC, which employs test inputs and the maximum difference of fitness values from multiple executions of the same test. We evaluate four alternative subsets of test input variables to train classifiers: 1) All input variables; 2) Excluding ambient conditions variables; 3) Excluding scene layout variables; 4) Excluding both ambient conditions and scene layout variables. To build the classifiers, we investigate the following widely-used techniques: Decision Trees (DT), Random Forests (RF), Support Vector Machines (SVM), and Multi-Layer Perceptrons (MLP) (Witten et al. 2011; Hagan et al. 1997).

We use the 1000 test inputs generated for PID, PYLOT and BEAMNG in RQ1-1 to develop training and test datasets for STEC and MTECclassifiers. We refer to the datasets corresponding to STEC and MTEC as D_{STEC} and D_{MTEC}, respectively. For D_{STEC}, we match each test input with the fitness value obtained from its first execution, resulting in one data point per test input. For D_{MTEC}, we couple each test input with a fitness value difference derived from two or more re-executions of that test input. We compute nine fitness differences per test input based on the ten consecutive re-executions of each test input. After removing duplicates, D_{MTEC} contains 4796 data points for PID, 3181 data points for PYLOT, and 6018 for BEAMNG.

To construct each data point in D_{STEC}, only a single test execution is required, resulting in a consistent cost for all data points in D_{STEC}. In contrast, data points in $D_{\text{MTEC}}$are obtained from a minimum of two to a maximum of ten re-executions, leading to varying costs associated with the data points in D_{MTEC}. We partition D_{MTEC} into nine subsets D^2_{MTEC} to D^{10}_{MTEC} where D^i_{MTEC} contains the fitness differences obtained from the first i executions. In other words, developing D^i_{MTEC} requires a consistent cost of i executions per data point. Since developing different D^i_{MTEC} partitions requires varying levels of effort, we assess the MTECclassifiers' performance based on D^i_{MTEC} datasets individually. Note that we train MTECclassifiers on the entire D_{MTEC}, but for testing, we apply them to D^2_{MTEC} to D^9_{MTEC} separately. We do not use D^{10}_{MTEC} as a test set since the effort required to build D^{10}_{MTEC} is equivalent to the effort needed for developing the ground truth (actual) labels, and the actual labels can be trivially derived for D^{10}_{MTEC}.

Finally, we compare our classifiers with a simple, non-ML baseline that labels a data point in D_{MTEC}flaky if the fitness difference associated to the data point is higher than the same threshold used for ground truth labeling, i.e., the threshold obtained as a soft flaky ratio of higher than 5%. Note that this baseline is only relevant to the D_{MTEC}datasets since it requires a fitness difference, and hence, at least two re-executions. We do not have any baselines, either from the literature or otherwise, that can operate on the D_{STEC}datasets.

Metrics We assess our classifiers using standard metrics, namely *Precision*, i.e., the ability of a classifier to precisely predict flaky test inputs, *Recall*, i.e., the ability of a classifier to predict all flaky test cases, and the *F1-Score*, i.e., the harmonic mean of precision and

recall (Goutte and Gaussier 2005). We used a 5-fold stratified cross-validation to ensure our models are trained and tested in a valid and unbiased way. For that, we allocated 80% of the data points for training and 20% for testing. Since our datasets might be imbalanced, we use synthetic minority oversampling technique (SMOTE) (Chawla et al. 2002) for balancing the data in the training set. We did not balance the test datasets to ensure that our model is only tested on the actual set of test inputs.

Results We developed and assessed 48 classifiers (4 classification techniques × 4 subsets of input variables × 3 setups) based on D_{STEC}. Table 7 shows the STECclassifiers with the highest F1-Score for each setup: For PID, DT with all variables; for PYLOT, DT with all but the scene layout variables; and for BEAMNG, RF with all but weather condition variables are the best STECclassifiers. In all three cases, the classifiers achieve high precision and recall values, i.e., recall more than 76% and precision more than 89%.

Similar to the above, we developed and assessed 48 MTECclassifiers, and selected the best for each test setup. For both PYLOT and PID, MLP with all variables, and for BEAMNG, RF with all variables yield the highest F1-Scores. Table 8 compares the precision, recall and F1-Score values of the baseline and the best MTECclassifiers obtained using test sets D^2_{MTEC} to D^9_{MTEC} for each setup. The second leftmost column of Table 8 shows the number of executions which corresponds to i in D^i_{MTEC}.

Both the baseline and the ground truth classify a datapoint as flaky when the fitness difference exceeds a specific threshold, utilizing the same threshold value. If the fitness difference exceeds the threshold after fewer than ten re-executions, it will also surpass the threshold when calculated from ten re-executions. Hence, the baseline's precision is consistently 100% and better than that of the MTECclassifiers. However, the baseline's recall is lower than the classifiers' recall for most cases. In particular, since ADS testing is time consuming, we are interested in the results for low-cost test sets, i.e. those built using four or fewer executions. In Table 8, we have highlighted the results for low-cost test sets in grey. MTECclassifiers notably outperform the baseline for low-cost test sets with a precision margin ranging from 5% to 38%. Specifically, for the least expensive test set (D^2_{MTEC}), MTECclassifiers yield F1-Score values that are 30%, 9% and 11% higher that those of the baseline for PID, PYLOT and BEAMNG, respectively. Full results for all the 96 STECand MTECclassifiers are available online (Online 2023).

The answer to RQ2-1 is that ML classifiers effectively identify flaky ADS test inputs with high precision and recall. Single-test-execution-based (STEC) classifiers achieve at least 89% precision and 76% recall across various test setups. Multi-test-execution-based (MTEC) classifiers, when applied to datasets requiring four or fewer test reruns, yield 95% or higher precision and 83% or higher recall, outperforming a non-ML baseline.

Table 7 The STEC classifiers with the highest F1-score

Setup	Inputs	Technique	Precision	Recall	F1-score
PID	All variables	DT	0.89	0.93	0.85
PYLOT	All variables w/o scene layout	DT	0.92	0.76	0.82
BEAMNG	All variables w/o weather	RF	1.00	0.93	0.96

Table 8 Comparing the best MTEC classifiers with our non-ML-baseline for different ADS test setups

Setup	# of Execs	Non-ML baseline			ML classifier		
		Precision	Recall	F1-score	Precision	Recall	F1-score
PID	2	1.00	0.37	0.54	0.97	0.75	**0.84**
	3	1.00	0.53	0.70	0.97	0.86	**0.91**
	4	1.00	0.66	0.79	0.97	0.91	**0.94**
	5	1.00	0.75	0.85	0.96	0.93	**0.95**
	6	1.00	0.82	0.90	0.96	0.96	**0.96**
	7	1.00	0.87	0.93	0.96	0.96	**0.96**
	8	1.00	0.92	0.96	0.96	0.98	**0.97**
	9	1.00	0.95	**0.97**	0.95	0.99	**0.97**
PYLOT	2	1.00	0.44	0.61	0.98	0.54	**0.70**
	3	1.00	0.60	0.75	0.96	0.73	**0.83**
	4	1.00	0.70	0.82	0.95	0.83	**0.89**
	5	1.00	0.77	0.87	0.94	0.88	**0.91**
	6	1.00	0.84	0.91	0.93	0.92	**0.92**
	7	1.00	0.90	**0.94**	0.90	0.94	0.92
	8	1.00	0.92	**0.96**	0.89	0.95	0.92
	9	1.00	0.96	**0.98**	0.88	0.97	0.92
BEAMNG	2	1.00	0.71	0.83	1.00	0.89	**0.94**
	3	1.00	0.86	0.92	0.99	0.93	**0.96**
	4	1.00	0.90	0.95	0.99	0.95	**0.97**
	5	1.00	0.93	0.96	1.00	0.96	**0.98**
	6	1.00	0.95	0.97	1.00	0.96	**0.98**
	7	1.00	0.97	**0.98**	0.99	0.97	**0.98**
	8	1.00	0.98	**0.99**	0.99	0.98	**0.99**
	9	1.00	0.99	**0.99**	0.99	0.98	**0.99**

4.6 RQ2-2 Results

This research question investigates if machine learning classifiers can improve the performance of ADS testing by requiring a limited number of test reruns. For this research question, we evaluate a random testing algorithm, denoted by RS_{ML}, that uses STECand MTECclassifiers according to Algorithm 3 to compute optimal fitness values with minimal test reruns. The algorithm uses STECpredictions after the first test execution and MTECpredictions after the subsequent test executions to determine whether, or not, a test is flaky and if the test re-executions should continue. We develop a baseline for RS_{ML}, denoted by RS_b, using the non-ML baseline from RQ2-1. Recall that the non-ML baseline could only be developed based on the D_{MTEC}datasets as we have no baseline that could operate based on the D_{STEC}datasets. The RS_b algorithm replaces STECin line 6 of Algorithm 3 with "true" as we have no alternative for STEC, and replaces MTECin line 15 of Algorithm 3 with the non-ML baseline.

We execute both RS_{ML} and RS_b for 50 iterations and for 20 times. We then compare their performance in obtaining optimal fitness values, while also recording the number of simulations (i.e., test executions) each algorithm performs. We set the maximum iterations for both RS_{ML} and RS_b to ten to be consistent with the $RS_{n=10}$ algorithm used for RQ1-3. In

Table 9 Number of simulations performed by RS_{ML} and RS_b for each test setup

	RS_b	RS_{ML}		RS_b	RS_{ML}		RS_b	RS_{ML}
PID	5323	4420	PYLOT	7391	3631	BEAMNG	4153	4396

RQ1-3, $RS_{n=10}$, which outperformed $RS_{n=1}$, executed 10, 000 simulations (50 iterations $\times$ 20 runs $\times$ 10 re-executions). We expect RS_{ML} and RS_b to obtain fitness values close to those obtained by $RS_{n=10}$ but using significantly fewer simulations.

The distributions of the best fitness values computed by RS_{ML} and RS_b are available online (Github paper 2023). Both RS_{ML} and RS_b obtain fitness values that are significantly more optimal than those obtained by $RS_{n=1}$, but less optimal than those obtained by $RS_{n=10}$. To compare RS_{ML} and RS_b, we provide the number of simulations each algorithm performs in Table 9. In addition, Table 10 statistically compares the distributions of fitness values obtained by RS_{ML} and RS_b for the four fitness functions of our three test setups. The results in Table 10 shows that for eight fitness functions, RS_{ML} obtains significantly better fitness functions than RS_b, while for four fitness functions, RS_b outperforms RS_{ML}. Overall and as shown in the fitness distributions available online (Online 2023), both algorithms obtain optimal fitness values that are quite close. However, as Table 9 shows, overall, RS_{ML} requires much fewer simulations to compute these optimal fitness values. Specifically, RS_{ML} requires 903 less simulations for PID and 3760 less simulations for PYLOT. Although RS_{ML} requires more simulations than RS_b for BEAMNG, the difference is relatively minor (only 243 additional simulations).

The answer to RQ2-2 is that our ADS random testing algorithm that uses ML classifiers achieves fitness values that are comparable to those of a non-ML baseline, but with significantly fewer simulations required.

4.7 Threats to Validity

The most important threats concerning the validity of our experiments are related to the internal, construct and external validity.

4.8 Internal Validity

To mitigate *internal validity* risks, which refer to confounding factors, we ensure that the PYLOT, TRAN, PID, and BEAMNG test setups share the same test input space and use consistent fitness functions. Our COMP setup uses the same test input space as the SBFT competition benchmark (Github cyber 2023) and maintains consistency with the benchmark by employing a single fitness function. We normalize the ranges of the fitness functions for all the setups before computing the soft flaky metric. In addition, we have used consistent thresholds for the fitness functions to identify failures and compute hard flakiness. In particular, for the lane keeping fitness function, F_1, we assume a lane invasion occurs whenever the ego car's distance to the center of lane increases 0.5 meters. For F_2 and F_3, we assume a collision occurs when the ego car's distance to other cars or static objects decreases 0.5 meters. For F_4, we assume that the ego car fails to reach its end goal when at the end of the simulation, the ego

Table 10 Statistical tests, Wilcoxon p-value and Vargha-Delaey $\hat{A}_{12}$, comparing the best fitness values obtained by RS_{ML} (abbreviated as ML) and RS_b (abbreviate as b) after performing the number of simulations in Table 9

Setup	p-value				A_{12}			
	F_1	F_2	F_3	F_4	F_1	F_2	F_3	F_4
PID	1.23e − 08 (b >ML)	2.37e − 09 (ML >b)	0.0003 (ML >b)	7.18e − 09 (b >ML)	0.29 (M)	0.78 (L)	0.54 (S)	0.31 (M)
PYLOT	8.00e − 10 (ML >b)	1.05e − 09 (ML >b)	0.0001 (ML >b)	7.12e − 10 (b >ML)	0.85 (L)	0.71 (L)	0.66 (M)	0.15 (L)
BEAMNG	2.10e − 07 (b >ML)	7.46e − 10 (ML >b)	7.51e − 10 (ML >b)	7.54e − 10 (ML >b)	0.11 (L)	0.88 (L)	0.59 (S)	0.59 (S)

car is more than one meter away from its final destination. These thresholds are consistent with those used by the original studies (Haq et al. 2022, 2023; Github cyber 2023).

For our experiments, we minimize, to the best of our abilities, internal randomness factors of CARLA and BeamNG that could be controlled through their random seeds or their configuration parameters. We identify potential sources of randomness in both, as per their official documentation (Foundations 2022a; Quick 2022b; BeamNG.tech 2023). Below, we outline the measures taken to control or mitigate these random elements.

In CARLA, we identify the traffic lights' function, traffic manager behaviour, and non-ego vehicles' behaviours as potential sources of randomness. We control traffic lights' behavior by fixing their initial states, and controlling the duration of red, green or yellow using a constant seed. As for the traffic manager, we set its random seed to a constant and ensure that it runs in deterministic mode by setting a the Boolean parameter that controls its mode of execution. In CARLA, each non-ego vehicle takes a random blueprint (i.e., type, shape, and size) when it is spawned. We fix the blueprint for each non-ego vehicle in our test inputs.

BeamNG does not provide any API to manipulate the blueprints of non-ego vehicles and the traffic lights. Further, its AI-based ADS does not react to traffic lights. Both BeamNG and CARLA may be executed in the synchronous or asynchronous modes. To reduce randomness, we use both in their synchronous mode where the client controls the simulator's updates. In contrast, in the asynchronous mode, the simulators run as fast as possible and handles client requests on the fly.

4.9 Construct Validity

Construct validity threats relate to the inappropriate use of metrics. Our hard flakiness metric is consistent with the notion of flaky tests for software code (Bell et al. 2018; Parry et al. 2021). For soft flakiness, the key attribute a metric should possess is the ability to capture the degree of variations in fitness function values across multiple re-runs. Our current choice for the SF metric, i.e, the difference between max and min, maintains the metric's scale consistent with that of the fitness function, facilitating easier interpretation. An alternative way to measure the SF metric is to use standard deviation. However, standard deviation presents a challenge due to its squared scaling of the fitness function values. In other words, if the differences between the fitness values and the mean exceed one, the standard deviation will be greater than the mean of these differences; and dually, if these differences are less than one, the standard deviation will be smaller. This scaling inconsistency complicates the interpretation of an SF metric measured by standard deviation, as the metric values do not align with the scale of fitness values.

4.10 External Validity

External validity is related to the generalizability of our results. The choice of our test setups may influence the generalizability of our results. Related to this threat, we note that we strive to diversify the setups used in our study in terms of the simulators, the ADS controllers and the test input designs. Specifically, our test setups are based on two widely-used simulators, four ADS controllers with different internal designs, and two different test input designs. Second, three of our test setups are adopted from the literature and have been previously used in ADS testing research (Gog et al. 2021; Github transfuser 2023; Github cyber 2023). Third, our aim is not to conclude a high frequency of flakiness for all ADS testing setups, but to study the prevalence of flaky tests in ADS simulation-based testing, their impact

on test results of ADS and ways to mitigate this impact. In particular, we believe that our proposed approach for assessing the prevalence, impact, and mitigation of flakiness in ADS testing maintains relevance and applicability across a wide range of simulators in the fields of robotics and cyber-physical systems. The above being said, our work would benefit from further experiments with a broader class of ADS test setups.

5 Discussion

In this section, we outline three key lessons derived from our findings in Tables 2 and 3 of RQ1-1. To better illustrate the connection between the lessons and our findings, we have summarized the hard and soft flaky rate ranges from Tables 2 and 3 in Table 11. Note that Table 11 shows ranges for non-negligible soft flaky rates, i.e., exceeding 5%. In addition to the three lessons, we present two clarifications concerning the scope of our study and its impact on prior research. Finally, we share observations on how our findings could influence future research in the field of simulation-based testing of ADS.

Lesson 1: The lane-keeping test setup shows the lowest rate of flaky tests The lane-keeping test setup, i.e., the COMP setup, consists of only the ego vehicle driving on a single-lane road and is focused on assessing the lane-keeping requirement, i.e., the *R1* requirement. As shown in Table 11, this setup has the lowest rate of flaky tests. Recall from Table 1 that the COMP setup shares the same ADS controller and the same simulator with the BEAMNG setup. However, BEAMNG and our three other setups feature a more extensive set of input variables. They encompass a map of a small town with a two-lane road intersection where other vehicles travel alongside the ego car, with several static objects also present in the environment. This richer test environment allows us to assess collision requirements, *R2* and *R3*, in addition to the lane keeping requirement. However, as Table 11 shows the restricted environment of the COMP setup leads to a considerably lower flaky test rate compared to the others.

Lesson 2: Modular DNN-based ADS reduces flaky tests compared to end-to-end DNN-based ADS As shown in Table 11, between the two CARLA-based setups with DNN-based ADS, i.e., PYLOT and TRAN, PYLOT yields lower flaky test rates. Given the test inputs and outputs consistency between these two setups and their shared simulator, we attribute the differences in their flaky rates to their ADS controller design. Across different re-executions, the simulator may pass slightly different images to the ADS due to synchronization and timing inconsistencies or due to the white noise addition function of the simulator. Both PYLOT and TRAN are DNN-enabled. But TRAN uses a single end-to-end transformer-based network, lacking PYLOT's modular structure. TRAN's outputs are produced by a single DNN,

Table 11 The hard and soft flaky rate ranges for our five test setups summarising the results from Tables 2 and 3

SETUP ID	HARD FLAKY RATE RANGES	SOFT FLAKY RATE RANGES (MORE THAN 5%)
PID	4% – 16%	5% – 65%
PYLOT	0% – 6%	4% – 23%
TRAN	0% – 40%	4% – 94%
BEAMNG	22% – 66%	47% – 68%
COMP	1%	12%

The table considers soft flaky rates that exceed 5%, as values below this threshold are deemed negligible

making the outputs potentially sensitive to minor input image variations. In contrast, PYLOT uses multiple DNNs, and also leverages an independent classical controller, i.e. MPC and PID (Gog et al. 2021). As a result, potential inaccuracies and noise in DNN outputs may be corrected by the controller, reducing the flaky test rate for the PYLOT setup with a modular DNN-based ADS.

Lesson 3: The Carla simulator yields a lower flaky test rate compared to the BeamNG simulator The PID and BEAMNG setups use their respective simulators' autopilots as ADS and have consistent inputs and outputs. Their difference is that PID is based on the Carla simulator, while BEAMNG is based on the BeamNG simulator. The considerably higher flaky test rate of the BEAMNG setup in Table 11 suggests that, provided with the same ADS and the same test input environment, the BeamNG simulator is more prone to producing flaky outputs compared to the Carla simulator.

Clarification of the scope: Identifying the root-causes of flakiness for simulators is out of the scope of our study Our research is focused on studying the presence and prevalence of non-determinism in ADS simulators and assessing the impact of this non-determinism on randomized ADS testing methods. More research is needed to understand flakiness causes in ADS simulators. Possible causes of flakiness include, among others, simulator bugs, simulator's autopilot instability, inconsistent timing and synchronization between the ADS and the simulator, as well as uncertainties in simulator's physical models. We are not able to assert which of these issues were the root causes of flakiness in the test setups discussed in this paper. However, in the case of our deep learning-based ADS models, i.e., Pylot and Transfuser, we have not detected any non-determinism during the inference phase. Hence, we believe that, in our experiments, the ADS itself is not a contributor to the observed flakiness. In other words, the DNN-based ADS under study in this paper, when used as a standalone component for inference purposes, are deterministic. We note that some root-causes of flakiness may be inevitable in this domain. Assuming that the simulator is bug-free, we have noticed that these issues may still cause non-determinism: (1) inconsistent timing and synchronization between the ADS and the simulator. Across different re-executions, the simulator may pass slightly different images or slightly different sensoray measurements to the ADS due to synchronization and timing inconsistencies. (2) Uncertainties in the simulatoin environment due to the presence of non-ego cars, traffic lights and pedestrians, etc. (3) White noise addition. Adding white noise to images is a common practice in data augmentation for machine learning and simulations, mimicking real-world disturbances (Foundations 2022a). If the simulator's images are used for training, the white noise addition helps robustify the ML models and improve their generalizability. In our experiments, we noticed that both Carla and BeamNG simulators add white noise to each frame. The white noise added by the simulator, however, may contribute to flakiness when we use simulators for testing.

Based on our observations, the root-causes of flakiness are related to the simulator or to the interactions between the ADS and the simulator. Hence, the flakiness predictors may not generalize beyond a simulator or even a pair of simulator and ADS. Therefore, in Section 4.5, we train a predictor for each pair of simulator and ADS.

In RQ1-2, we identified three different types of variations in the flaky simulations that we checked manually. Based on our observations, the type I and II variations can be attributed to the simulator, while type III variations are likely a result of synchronization timing inconsistencies between the simulator and the ADS or are due to the white noise addition. This observation is indeed consistent with the results shown in Table 4. Specifically, Table 4 shows that the PID setup, which uses the simulator's PID controller as ADS, exhibits the

observed type I and II variations. This supports our hypothesis that the simulator's autopilot, specifically Carla PID, is the source of these inconsistencies. Conversely, most type III inconsistencies are related to the PYLOT setup, which uses an external DNN-based ADS, and is prone to issues caused by the timing inconsistencies between the ADS and the simulator and the white noise addition.

Clarification of the impact: Our study does not invalidate the comparison results of prior simulation-based ADS testing research, but the absolute values of metrics in these studies may be impacted by flakiness Previous research on simulation-based ADS testing typically involves re-running proposed test strategies for different sampled inputs and conducting statistical comparisons. For studies focused on comparing different test strategies, this approach accounts for the test strategy's randomness and the potential non-determinism in ADS simulators. While the relative comparison of metrics remains unaffected, the absolute values of metrics, such as the number of failure scenarios and optimal fitness values, can be impacted by simulator flakiness. This indicates a need to revisit the metrics and assessment methods for ADS testing research, which we will discuss subsequently.

Implications of our results for future research on simulation-based ADS testing Considering the engineering effort needed to set up a simulator and integrate it with an ADS for testing purposes, research papers should place more emphasis on clearly detailing and characterizing the integration process and test input specifications. Currently, greater focus is placed on devising different heuristics for test strategies, while ADS simulators are often treated as black boxes. Given a test input design, the degree of potential randomness and non-determinism incurred by a simulator used for testing should be explicitly measured and reported.

To perform ADS testing, we are faced with a spectrum of possibilities for configuring ADS test setups. On one extreme, we may consider a complex urban map with an arbitrary number of non-ego vehicles and pedestrians. This setup, while allowing us to test ADS for a variety of safety requirements and situations, likely leads to a significant flaky test rate. On another extreme, we may consider a restricted map with no, or few, fixed-behavior and controlled mobile objects other than the ego vehicle. This setup, while being restricted, likely has a low or negligible flaky test rate. While fitness values of individual scenarios can be a good measure of test progress and identification of failures for restricted setups, they might be insufficient for relaxed setups due to non-determinism. These findings are consistent with recent studies on misconceptions in DNN testing (Zohdinasab et al. 2023; Riccio and Tonella 2023). Therefore, an interesting research direction is to develop metrics and evaluation methods that remain reliable in the presence of simulator non-determinism.

6 Related Work

Recent research on flaky tests in software code-bases reveals their notable prevalence in both commercial and open-source contexts. Google reported that nearly 16% of their 4.2 million test cases are flaky (Micco 2018), while 26% of 3,871 distinct builds sampled from Microsoft's system failed due to flakiness (Parry et al. 2021). The Microsoft Windows and Dynamics teams estimated a 5% rate of flaky test failures (Herzig and Nagappan 2015), while the Randoop repository showed a similar rate of 5% flaky tests for its open-source Java projects (Paydar and Azamnouri 2019). Our results in RQ1-1 reveal that the hard flaky test rates for the three ADS test setups adopted from the literature, namely PYLOT (Haq et al. 2022), TRAN (Github transfuser 2023), COMP (Github cyber 2023), are 6%, 32%, and

1% respectively for at least one of their fitness functions. Overall, between 4% and 68% of the generated tests across our five test setups exhibited noticeable variations in their fitness values. These results indicate that flakiness in ADS testing is comparable to flakiness in software code-bases.

Two recent simulation-based ADS testing studies have briefly noted the presence of flaky tests (Birchler et al. 2023; Nguyen et al. 2021). One study, based on the COMP setup, reported 1% to 5% flaky tests that were excluded from the results (Birchler et al. 2023). The other study, based on the SVL simulator (Github svl 2023), mitigated flakiness by voiding traffic lights (Nguyen et al. 2021). We show that when test inputs and outputs are complex, e.g., they include the elements in Fig. 4, flaky tests can be prevalent. In such situations, as shown in RQ1-3, accounting for flakiness by rerunning tests significantly improves the results of randomized testing algorithms.

Detecting flaky tests often involves rerunning tests multiple times, which can be costly and time-consuming (Parry et al. 2021). Some methods approximate flakiness without requiring multiple test reruns by leveraging execution history and coverage information (Bell et al. 2018; Shi et al. 2016). Some of these approaches rely on ML, NLP or probabilistic techniques to enhance their effectiveness in identifying flaky tests (Dutta et al. 2020; Alshammari et al. 2021; Bell et al. 2018). To our knowledge, no prior work has studied the presence, impact, or cost-effective prediction of flakiness in ADS testing. In RQ2-1 and RQ2-2, we demonstrate that ML classifiers can effectively identify flaky tests with a limited number of test reruns.

7 Conclusion

This paper presents the first study evaluating the impact of flaky simulators on testing Autonomous Driving Systems (ADS). Our study includes combinations of two widely-used ADS simulators, CARLA and BeamNG, and three different ADS types. Our study shows that flakiness is a common occurrence in ADS simulation-based testing. We demonstrate that ML classifiers trained for each test setup are able to identify flaky ADS tests, requiring only a single run and achieving F1-scores of 85%, 82% and 96% for three different ADS test setups. Considering the widespread occurrence of flaky tests in simulation-based ADS testing, it is crucial to evaluate the flaky test rates. If they prove to be significant, implementing mitigation strategies like test repetition or using ML classifiers, as demonstrated in this paper, can be beneficial. Alternatively, it is possible to assess test results using metrics that remain robust in the presence of ADS simulator flakiness.

Our study is unique in terms of the diversity of ADS test setups. To the best of our knowledge, very few studies on ADS testing are performed on two or more simulators and three different ADS types. Nonetheless, further research is needed to understand the causes of flakiness in ADS simulators and to establish methods for assessing ADS testing algorithms given simulators' non-determinism.

Acknowledgements We gratefully acknowledge the financial support received from NSERC of Canada through their Discovery program.

Data Availability Our online material include: (1) a complete description and implementation of our test generators including test inputs, fitness functions and thresholds, (2) scripts for reproducing our results, and (3) raw datasets for the experiments (Github paper 2023). Complementary experiment results, diagrams and statistical tests are also included in the online material (Online 2023).

Declarations

COI Statement The authors declared that they have no conflict of interest.

References

(2016) Udacity self-driving challenge 2. https://github.com/udacity/self-driving-car/tree/master/challenges/challenge-2, Accessed 11 Oct 2019
(2022a) Foundations. https://carla.readthedocs.io/en/latest/foundations/. Accessed 15 Nov 2022
(2022b) Quick start. https://carla.readthedocs.io/en/latest/start_quickstart/. Accessed 15 Nov 2022
(2022) Raquel Urtasun's tech company develops self-driving vehicle simulator. https://www.thestar.com/business/2022/02/09/raquel-urtasuns-tech-company-develops-self-driving-vehicle-simulator.html. Accessed May 2022
(2023) BeamNG.tech Website. https://beamng.tech. Accessed 3 Mar 2023
(2023) Carla Challenge. https://carla.readthedocs.io/en/latest/adv_traffic_manager/, Accessed 1 Feb 2023
(2023) Github repo for cyber-physical systems testing tool competition. https://github.com/sbft-cps-tool-competition/cps-tool-competition, Accessed 10 Apr 2023
(2023) Github repo for svl simulator: an autonomous vehicle simulator. https://github.com/lgsvl/simulator, Accessed 10 Apr 2023
(2023) Github repo for the paper. https://github.com/anonoymous9423013/anonymous_paper/. Accessed 10 Apr 2023
(2023) Github repo for transfuser: imitation with transformer-based sensor fusion for autonomous driving. https://github.com/autonomousvision/transfuser. Accessed 10 Apr 2023
(2023) Online supplementary material for the paper. https://github.com/anonoymous9423013/anonymous_paper/tree/main/supplementary_materials. Accessed 26 Apr 2023
Abdessalem RB, Nejati S, Briand LC, Stifter T (2018) Testing vision-based control systems using learnable evolutionary algorithms. In: 2018 IEEE/ACM 40th international conference on software engineering (ICSE), IEEE, pp 1016–1026
Afzal A, Katz DS, Le Goues C, Timperley CS (2021) Simulation for robotics test automation: developer perspectives. In: 2021 14th IEEE conference on software testing, verification and validation (ICST), pp 263–274
Ahlgren J, Bojarczuk K, Drossopoulou S, Dvortsova I, George J, Gucevska N, Harman M, Lomeli M, Lucas SM, Meijer E, et al (2021) Facebook's cyber–cyber and cyber–physical digital twins. In: Evaluation and assessment in software engineering, pp 1–9
Alshammari A, Morris C, Hilton M, Bell J (2021) Flakeflagger: predicting flakiness without rerunning tests. In: 43rd IEEE/ACM international conference on software engineering: companion proceedings, ICSE Companion 2021, Madrid, Spain, May 25-28, 2021, IEEE, p 187
Bell J, Legunsen O, Hilton M, Eloussi L, Yung T, Marinov D (2018) Deflaker: automatically detecting flaky tests. In: 2018 IEEE/ACM 40th international conference on software engineering (ICSE), pp 433–444
Birchler C, Khatiri S, Bosshard B, Gambi A, Panichella S (2023) Machine learning-based test selection for simulation-based testing of self-driving cars software. Empir Softw Eng 28(3):71
Borg M, Abdessalem RB, Nejati S, Jegeden F, Shin D (2021) Digital twins are not monozygotic - cross-replicating ADAS testing in two industry-grade automotive simulators. In: 14th IEEE conference on software testing, verification and validation, ICST 2021, Porto de Galinhas, Brazil, April 12-16, 2021, IEEE, pp 383–393
Capon JA (1991) Elementary Statistics for the Social Sciences: Study Guide. Wadsworth Publishing Company, Belmont, CA, USA
Chawla NV, Bowyer KW, Hall LO, Kegelmeyer WP (2002) Smote: synthetic minority over-sampling technique. J Artif Intell Res 16:321–357
Dosovitskiy A, Ros G, Codevilla F, Lopez A, Koltun V (2017) CARLA: an open urban driving simulator. In: Proceedings of the 1st annual conference on robot learning, pp 1–16
Dutta S, Shi A, Choudhary R, Zhang Z, Jain A, Misailovic S (2020) Detecting flaky tests in probabilistic and machine learning applications. In: Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis, association for computing machinery, New York, USA, ISSTA 2020, pp 211–224, https://doi.org/10.1145/3395363.3397366

Gog I, Kalra S, Schafhalter P, Wright MA, Gonzalez JE, Stoica I (2021) Pylot: a modular platform for exploring latency-accuracy tradeoffs in autonomous vehicles. In: 2021 IEEE international conference on robotics and automation (ICRA), IEEE, pp 8806–8813

Goutte C, Gaussier E (2005) A probabilistic interpretation of precision, recall and f-score, with implication for evaluation. In: Losada DE, Fernández-Luna JM (eds) Advances in Information Retrieval. Springer, Berlin, Heidelberg, pp 345–359

Hagan MT, Demuth HB, Beale M (1997) Neural network design. PWS Publishing Co

Haq FU, Shin D, Nejati S, Briand LC (2020) Comparing offline and online testing of deep neural networks: An autonomous car case study. In: 13th IEEE international conference on software testing, validation and verification, ICST 2020, Porto, Portugal, October 24-28, 2020, IEEE, pp 85–95

Haq FU, Shin D, Nejati S, Briand LC (2021) Can offline testing of deep neural networks replace their online testing? Empir Softw Eng 26(5):90

Haq FU, Shin D, Briand L (2022) Efficient online testing for dnn-enabled systems using surrogate-assisted and many-objective optimization. In: 2022 IEEE/ACM 44th international conference on software engineering (ICSE), pp 811–822, https://doi.org/10.1145/3510003.3510188

Haq FU, Shin D, Briand LC (2023) Many-objective reinforcement learning for online testing of dnn-enabled systems. In: 45th IEEE/ACM international conference on software engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023, IEEE, pp 1814–1826

Harman M, McMinn P (2010) A theoretical and empirical study of search-based testing: local, global, and hybrid search. IEEE Trans Softw Eng 36(2):226–247. https://doi.org/10.1109/TSE.2009.71

Herzig K, Nagappan N (2015) Empirically detecting false test alarms using association rules. In: 2015 IEEE/ACM 37th IEEE international conference on software engineering, vol 2, pp 39–48

Luke S (2013) Essentials of Metaheuristics, 2nd edn. Lulu, available for free at http://cs.gmu.edu/~sean/book/metaheuristics/

Luo Q, Hariri F, Eloussi L, Marinov D (2014) An empirical analysis of flaky tests. In: Cheung S, Orso A, Storey MD (eds) Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering, (FSE-22), Hong Kong, China, November 16 - 22, 2014, ACM, pp 643–653

Matinnejad R, Nejati S, Briand LC (2017) Automated testing of hybrid simulink/stateflow controllers: industrial case studies. In: Bodden E, Schäfer W, van Deursen A, Zisman A (eds) Proceedings of the 2017 11th joint meeting on foundations of software engineering, ESEC/FSE 2017, Paderborn, Germany, September 4-8, 2017, ACM, pp 938–943

Micco J (2018) Advances in continuous integration testing at google

Nguyen V, Huber S, Gambi A (2021) Salvo: automated generation of diversified tests for self-driving cars from existing maps. In: 2021 IEEE international conference on artificial intelligence testing (AITest), pp 128–135

Parry O, Kapfhammer GM, Hilton M, McMinn P (2021) A survey of flaky tests. ACM Trans Softw Eng Methodol 31(1), https://doi.org/10.1145/3476105

Paydar S, Azamnouri A (2019) An experimental study on flakiness and fragility of randoop regression test suites. In: Fundamentals of software engineering

Riccio V, Tonella P (2023) When and why test generators for deep learning produce invalid inputs: an empirical study. In: 45th IEEE/ACM international conference on software engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023, IEEE, pp 1161–1173

Samak CV, Samak TV, Kandhasamy S (2020) Control strategies for autonomous vehicles. arXiv:2011.08729

Shi A, Gyori A, Legunsen O, Marinov D (2016) Detecting assumptions on deterministic implementations of non-deterministic specifications. In: 2016 IEEE international conference on software testing, verification and validation (ICST), pp 80–90

Ulbrich S, Menzel T, Reschka A, Schuldt F, Maurer M (2015) Defining and substantiating the terms scene, situation, and scenario for automated driving. In: 2015 IEEE 18th international conference on intelligent transportation systems, pp 982–988, https://doi.org/10.1109/ITSC.2015.164

Vargha A, Delaney HD (2000) A critique and improvement of the cl common language effect size statistics of mcgraw and wong. J Educ Behav Stat 25(2):101–132

Witten IH, Frank E, Hall MA (2011) Data mining: practical machine learning tools and techniques, 3rd edn. Morgan Kaufmann Series in Data Management Systems, Morgan Kaufmann, Amsterdam

Zeller A, Gopinath R, Böhme M, Fraser G, Holler C (2023) Code coverage. In: The Fuzzing Book, CISPA Helmholtz Center for Information Security, https://www.fuzzingbook.org/html/Coverage.html, retrieved 2023-01-07 13:54:15+01:00

Zhong Z, Kaiser G, Ray B (2023) Neural network guided evolutionary fuzzing for finding traffic violations of autonomous vehicles. IEEE Trans Softw Eng 49(4):1860–1875. https://doi.org/10.1109/TSE.2022.3195640

Zohdinasab T, Riccio V, Gambi A, Tonella P (2023) Deephyperion: Exploring the feature space of deep learning-based systems through illumination search. In: Engels G, Hebig R, Tichy M (eds) Software Engineering 2023, Fachtagung des GI-Fachbereichs Softwaretechnik, 20.-24. Februar 2023, Paderborn, Gesellschaft für Informatik e.V., LNI, vol P-332, pp 131–132

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.